

Exercício-Programa 1 - Agente de Busca para o Pac-Man

Prazo limite de entrega no e-Disciplinas: 10/04/2026 às 23:59

1 Introdução

Neste exercício-programa estudaremos a abordagem de construção de agentes inteligentes capazes de resolver problemas através de uma busca no espaço de estados do jogo [Pac-Man](#). Utilizaremos parte do material/código livremente disponível do curso UC Berkeley CS188 ¹.

Os objetivos deste exercício-programa são:

- (i) compreender a abordagem de resolução de problemas baseada em busca no espaço de estados, através da construção de um jogador autônomo de Pac-Man;
- (ii) implementar algoritmos de busca informada e não-informada, bem como um algoritmo de busca online, e comparar seus desempenhos.

Cenário 1 - Ponto fixo de comida

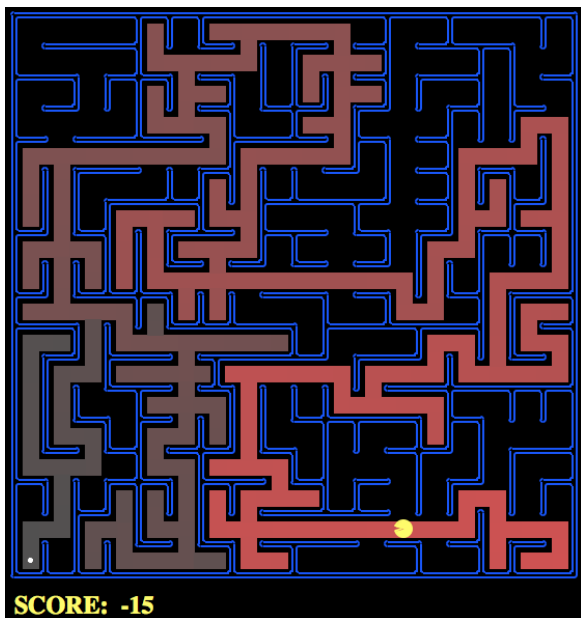


Figura 1: Exemplo de um problema em que o Pac-man precisa encontrar um caminho até uma comida. O tabuleiro do jogo mostra os estados visitados em uma DFS por cor, isto é, quanto mais vermelho escuro mais cedo na busca o estado foi visitado.

Cenário 2 - Cantos do labirinto

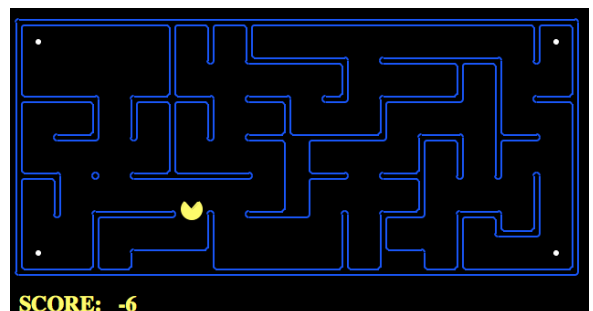


Figura 2: Exemplo de um problema em que o Pac-man precisa encontrar um caminho que passe pelas comidas que estão nos 4 cantos do labirinto.

¹http://ai.berkeley.edu/project_overview.html

2 Pac-Man como problema de busca

Nesse exercício-programa você resolverá diversos problemas de busca. Independente da busca, a interface que implementa a formulação do problema é definida pela classe abstrata e seu conjunto de métodos abaixo (disponível no arquivo `search.py` na pasta **search/**):

```
# arquivo search.py

class SearchProblem:

    def getStartState(self):
        """ Returns the start state for the search problem. """
        # ...

    def isGoalState(self, state):
        """ Returns True if and only if the state is a valid goal state. """
        # ...

    def getSuccessors(self, state):
        """ For a given state, this should return a list of triples
        (successor, action, stepCost), where 'successor' is a successor to
        the current state, 'action' is the action required to get there, and
        'stepCost' is the incremental cost of expanding to that successor. """
        # ...

    def getCostOfActions(self, actions):
        """ This method returns the total cost of a particular sequence
        of actions. The sequence must be composed of legal moves. """
        # ...
```

Veja que os métodos acima são semelhantes a algumas das funções usadas nos slides da Aula 4 disponível no e-disciplinas:

- `problem.getStartState()` é equivalente a `problema.ESTADO.INICIAL` nos slides.
- `problem.isGoalState(state)` é equivalente a `problema.TESTE.META(nó.ESTADO)` nos slides.
- `problem.getSuccessors(state)` é semelhante a função `Expand(node, problem)` nos slides, mas note que `problem.getSuccessors(state)` devolve uma lista de triplas (`estado_sucessor`, `ação`, `custo`), e não uma lista de nós sucessores.

As ações do Pac-Man são apenas quatro, uma para cada direção de movimento do ambiente *grid*: `Directions.NORTH`, `Directions.SOUTH`, `Directions.EAST` e `Directions.WEST`. Se ao mover para uma célula ele encostar em uma comida, essa comida é coletada automaticamente. Uma ação se torna impossível quando há uma parede na célula vizinha na direção do movimento. Todas as ações têm custo unitário.

No cenário 1, que corresponde à classe `PositionSearchProblem` no arquivo `searchAgents.py`, o estado é apenas uma *célula*, representada por um *par de coordenadas inteiras*, indicando a posição do agente. Ela ignora as posições das comidas e fantasmas pois não são necessárias para esse problema. Já na classe `CornersProblem`, o estado (que você definirá na parte 4.2) precisará conter informações importantes sobre o caminho para que a busca funcione.

3 Implementação

Arquivos que você precisará editar:

- **search/search.py** onde os algoritmos de busca serão implementados;
- **search/searchAgents.py** onde os agentes baseados em busca serão implementados, isto é, onde formulamos os problemas que resolveremos nesse EP.

Observação: Utilize as estruturas de dados **Stack**, **Queue** e **PriorityQueue** disponíveis no arquivo **util.py** para implementação da fronteira de busca. Essas implementações são necessárias para manter a compatibilidade com o arquivo de testes (**autograder.py**). Listas, tuplas, tuplas nomeadas, conjuntos e dicionários do Python podem ser utilizados sem problemas caso necessário.

Os métodos das estruturas de dados são:

- **util.Stack():**
 - **push(item):** adiciona um item no topo da pilha.
 - **pop():** remove o item no topo da pilha e devolve o item removido.
 - **isEmpty():** devolve **True** se a pilha estiver vazia, e devolve **False** caso contrário.
- **util.Queue():**
 - **push(item):** adiciona um item no final da fila.
 - **pop():** remove o item na frente da fila e devolve o item removido.
 - **isEmpty():** devolve **True** se a fila estiver vazia, e devolve **False** caso contrário.
- **util.PriorityQueue():**
 - **push(item, priority):** adiciona um novo item na fila com a prioridade passada como argumento do método.
 - **pop():** remove o item na fila com a menor prioridade e devolve o item removido.
 - **isEmpty():** devolve **True** se a fila estiver vazia, e devolve **False** caso contrário.
 - **update(item, priority):** se o item já estiver na fila com prioridade maior do que a passada nos argumentos, apenas atualiza o valor da prioridade sem adicionar um item novo na fila. Se o item já estiver na lista mas com prioridade menor do que a passada nos argumentos, nada é feito. Se o item não estiver na fila, este método age igual a uma chamada de **push(item, priority)**.

4 Parte prática

Neste EP, você deverá resolver diferentes problemas de busca para dois ambientes diferentes do jogo Pac-Man, que chamaremos de Cenário 1 e Cenário 2. No Cenário 1 o Pac-Man deve procurar a comida que está localizada em uma única posição do labirinto. No Cenário 2, o Pac-Man deve procurar a comida que está distribuída nos quatro cantos do labirinto. Você deverá implementar os seguintes algoritmos de busca: Busca em Largura, Busca em profundidade, Busca de Custo Uniforme, Busca Heurística e um algoritmo de Busca Online chamado de LRTA*. Para isso, você deverá implementar algumas funções nos arquivos **search.py** e **searchAgents.py** (sinalizadas pelo comentário ***** ADD YOUR CODE HERE *****), contendo um exemplo de código que deverá ser eliminado, chamado de o código **util.raiseNotDefined()**.

Observação: Não esqueça de remover o código **util.raiseNotDefined()** ao final de cada função.

4.1 Cenário 1 - Encontrando a comida numa única localização do labirinto

Para o Cenário 1, a formulação do agente de busca já está disponível no arquivo **searchAgents.py** na classe **PositionSearchProblem**, isto é, a definição da tupla $\langle s_0, S, A, T, g, M \rangle$ define o agente de busca para esse ambiente.

Código 1 - Busca em Profundidade (DFS)

Implemente a busca em profundidade (DFS) no arquivo **search.py** na função **depthFirstSearch**. Para que a busca seja completa, implemente busca em grafo, que evita a expansão de estados previamente visitados ou já pertencentes à borda. Para testar seu algoritmo rode os comandos:

```
$ python pacman.py -l tinyMaze -p SearchAgent
$ python pacman.py -l mediumMaze -p SearchAgent
$ python pacman.py -l bigMaze -z .5 -p SearchAgent
```

Observação: note que, assim como pode ser visto na Figura 1, o tabuleiro do jogo mostra os estados visitados pela busca por cor, isto é, quanto mais vermelho escuro mais cedo na busca o estado foi visitado (o que serve para ilustrar e visualizar a árvore de busca). Além disso, para todas essas buscas seu algoritmo DFS deve encontrar uma solução rapidamente, isto é, se demorar mais que alguns milissegundos, algo provavelmente está errado no seu código!

Código 2 - Busca em Largura (BFS)

Implemente a busca em largura (BFS) no arquivo **search.py** na função **breadthFirstSearch**. Novamente, implemente busca em grafo. Para testar seu algoritmo rode os comandos:

```
$ python pacman.py -l mediumMaze -p SearchAgent -a fn=bfs
$ python pacman.py -l bigMaze -p SearchAgent -a fn=bfs -z .5
```

Observação: se o Pac-Man se mover muito devagar, tente usar a opção **--frameTime 0** na linha de comando.

Código 3 - Busca de Custo Uniforme

Implemente a busca de custo uniforme (UCS) no arquivo **search.py** na função **uniformCostSearch**. Para isso, você precisa adicionar custos variados às ações do agente UCS. No arquivo **searchAgents.py** estão disponíveis 2 implementações de agentes com custo variado (que dependem da posição do Pac-Man): **StayEastSearchAgent** e **StayWestSearchAgent**. Teste esse algoritmo para esses dois agentes. Novamente, implemente o UCS como busca em grafo. Para testar seu algoritmo, rode o comando:

```
$ python pacman.py -l mediumMaze -p SearchAgent -a fn=ucs
$ python pacman.py -l mediumDottedMaze -p StayEastSearchAgent
$ python pacman.py -l mediumScaryMaze -p StayWestSearchAgent
```

Código 4 - Busca heurística (A^*)

Implemente busca em grafo A^* no arquivo `search.py` na função `aStarSearch`. Para testar seu algoritmo rode o comando:

```
$ python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic
```

Código 5 - Busca de Aprofundamento Iterativo (ID-DFS)

Esse exercício programa é obrigatório para os alunos de pós-graduação e optativo para os alunos da graduação, valendo um bônus de 1 ponto na média final.

Implemente a busca de aprofundamento iterativo (ID-DFS) na função `iterativeDeepeningSearch` no arquivo `search.py`. **Lembre-se que você deve fazer uma busca em grafo.** Lembrando que, para que a busca ID-DFS em grafo continue ótima (como é no caso de busca em árvore), é necessária uma pequena modificação: um nó filho é inserido na borda somente se o estado correspondente não já está na borda ou em explorado. Use como base o algoritmo da Aula 4 (Busca Cega), slide 87 (Note que a versão simplificada do ID-DFS (slide 110) é de busca em árvore). Para testar seu algoritmo rode os comandos:

```
$ python pacman.py -l mediumMaze -p SearchAgent -a fn=iddfs
$ python pacman.py -l bigMaze -p SearchAgent -a fn=iddfs -z .5
```

Observação: Você pode comparar seu algoritmo ID-DFS com o algoritmo BFS para verificar a otimalidade da solução e incluir no relatório.

Código 6 - Busca heurística com Aprendizado em Tempo Real (LRTA*)

Esse exercício programa é obrigatório para os alunos de pós-graduação e optativo para os alunos da graduação, valendo um bônus de 1 ponto na média final.

Nesse item você deverá implementar o algoritmo LRTA* (Learning Real-Time A*). Você deverá se basear no algoritmo da Figura 2 na Seção 6.1. do artigo **Learning in Real-Time Search: A Unifying Framework**, disponível em <https://arxiv.org/pdf/1110.4076>. Implemente a busca (LRTA*) no arquivo `search.py` na função `lrtaStarSearch`. Teste o algoritmo para cada tamanho de *layout* do Pac-Man com números de *trials* 10, 20 e 100. Use *lookahead* de 1.

Reporte em uma tabela para cada configuração (*layout*, número de *trials*) os seguintes resultados:

1. custo total do caminho
2. número de nós expandidos; e
3. estimativa final do custo do estado inicial no último *trial*.

Para testar seu algoritmo rode o comando:

```
$ python pacman.py -l smallMaze -p SearchAgent -a fn=lrta,heuristic=manhattanHeuristic
$ python pacman.py -l mediumMaze -p SearchAgent -a fn=lrta,heuristic=manhattanHeuristic
$ python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=lrta,heuristic=manhattanHeuristic
```

Observação: Você pode passar a opção `-q` (ou `--quietTextGraphics`) para não iniciar a simulação gráfica do algoritmo.

4.2 Cenário 2 - Encontrando os cantos com comida do labirinto

Para este cenário, vamos executar e analisar os algoritmos implementados anteriormente, sem a necessidade de modificá-los. A única modificação que você deverá fazer é definir uma nova função heurística para o A^* , como mencionado a seguir.

Código 7 - Formulação de problema de busca dos 4 cantos

Nesse exercício vamos implementar um novo problema de busca. Você deverá completar a implementação dos métodos da classe `CornersProblem` no arquivo `searchAgents.py`. Você deverá escolher uma representação de estados que codifique **somente a informação necessária** para detectar se todos os 4 cantos do labirinto foram visitados. Nessa classe, a inicialização já fornece duas variáveis de instância com informações úteis:

- `self.walls`: Uma matriz booleana tal que `self.walls[x][y] = True` se tem uma parede na célula (x, y) .
- `self.corners`: Uma lista de pares (x, y) que indica as coordenadas dos cantos do labirinto.
- `self.startingPosition`: A célula (x, y) em que o agente está.

Para testar sua formulação do problema, rode o comando:

```
$ python pacman.py -l tinyCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
$ python pacman.py -l mediumCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

Observação: Não utilize um objeto `GameState` como estado de busca! Se utilizar, seu código provavelmente ficará errado e muito lento.

Código 8 - Heurística para o problema dos 4 cantos

Além da nova formulação de problema, você deverá implementar uma heurística não trivial e consistente na função `cornersHeuristic`. Para testar sua heurística rode o comando:

```
$ python pacman.py -l mediumCorners -p AStarCornersAgent -z 0.5
```

Nesse exercício consideramos heurísticas triviais aquelas que devolvem 0 (zero). A heurística trivial não ajuda muito do ponto de vista da eficiência do algoritmo. Você pode usar a heurística trivial para compará-la com a sua proposta de heurística não-trivial.

Dica: A função `util.manhattanDistance(xy1, xy2)` estima a distância entre duas células e pode ser usada para criar uma heurística.

O algoritmo $LRTA^*$ pode ser testado para esse problema rodando o comando:

```
$ python pacman.py -l mediumCorners -p LRTAStarCornersAgent -z 0.5
```

5 Relatório

Após o desenvolvimento da parte prática, você deverá testar seus algoritmos e redigir um relatório, claro e sucinto (máximo de 4 páginas), fazendo uma análise comparativa de desempenho dos algoritmos implementados nos dois cenários de teste. Assim, você deverá:

- (a) compilar em tabelas as estatísticas das buscas implementadas em termos de nós expandidos e tamanho de plano;
- (b) discutir as vantagens e desvantagens de cada método, no contexto dos dados obtidos;
- (c) responder as questões teóricas;
- (d) sugerir possíveis melhorias ao sistema e relatar dificuldades.

5.1 Questões

Questão 1 - Busca em Profundidade. Na classe `PositionSearchProblem`, podemos observar que os sucessores são gerados na seguinte ordem:

1. `Directions.NORTH`
2. `Directions.SOUTH`
3. `Directions.EAST`
4. `Directions.WEST`

Observe as execuções dos 3 problemas (`tinyMaze`, `mediumMaze`, `bigMaze`) no cenário 1. A ordem de exploração do espaço de estados seguiu conforme esperado? O tamanho da solução encontrada para cada um dos problemas está de acordo com essa ordem de exploração? O que aconteceria se tivéssemos implementado uma busca DFS em árvore? Justifique suas respostas.

Questão 2 - Busca em Largura. A sua implementação de busca em largura encontra uma solução de custo mínimo? Se alterarmos a ordem de geração dos estados sucessores, a BFS encontraria soluções diferentes? Justifique suas respostas.

Questão 3 - Busca de Custo Uniforme. Execute os comandos abaixo e explique sucintamente o comportamento dos agentes `StayEastSearchAgent` e `StayWestSearchAgent` em termos da função custo utilizada por cada agente.

```
$ python pacman.py -l mediumDottedMaze -p StayEastSearchAgent
$ python pacman.py -l mediumScaryMaze -p StayWestSearchAgent
```


Para justificar suas respostas, você pode levar em consideração os diferentes valores da função custo de acordo com as definições das classes acima, a saber: dada a posição (x, y) do Pac-Man, a classe `StayEastSearchAgent` define o custo de uma ação como $(0.5)^x$; e a classe `StayWestSearchAgent` define o custo como 2^x .

Questão 4 - Busca Heurística. Você deve ter percebido que o algoritmo A^* encontra uma solução mais rapidamente que outras buscas. Por quê? Qual a razão para se implementar uma heurística consistente para sua implementação da busca A^* ?

Questão 5 - Busca Heurística LRTA* (somente para os alunos de pós-graduação). Compare os resultados obtidos para os testes do LRTA*. Quais características diferenciam esse algoritmo dos demais algoritmos implementados? O que você pode dizer sobre o valor estimado do estado inicial a medida que o número de *trials* aumenta? Com base somente nos resultados obtidos como você pode verificar se o algoritmo convergiu?

6 Entrega

Você deve entregar **dois arquivos** separados:

- (1) Um arquivo zip chamado `EP1-NUSP-NOME-COMPLETO.zip` contendo **APENAS** os arquivos `se-arch.py` e `searchAgents.py` com as implementações da parte prática;
- (2) Um arquivo chamado `rel-EP1-NUSP-NOME-COMPLETO.pdf` em formato PDF contendo o relatório com as questões, comparações e discussão dos resultados (máximo de 4 páginas).

Não esqueça de identificar cada arquivo com seu nome e número USP! Faça isso tanto no código (coloque um cabeçalho em forma de comentário) quanto no relatório (coloque em destaque no título).

7 Critério de avaliação

O critério de avaliação atribuirá pesos iguais à parte prática e relatório. A avaliação da parte prática dependerá parcialmente dos resultados dos testes automatizados do `autograder.py`. Dessa forma você terá como avaliar por si só parte da nota que receberá para a parte prática. **Note que para os algoritmos LRTA* e ID-DFS não há testes automatizados no autograder.** Avaliaremos esse algoritmo separadamente. Para rodar os testes automatizados, execute o seguinte comando:

```
$ python autograder.py
```

Com relação ao relatório, avaliaremos principalmente sua forma de interpretar comparativamente os desempenhos de cada busca. Não é necessário detalhar a estratégia de cada busca ou qual estrutura de dados você utilizou para cada busca, mas deve ficar claro que você compreendeu os resultados obtidos conforme esperado dado as características teóricas de cada busca.

Pontuação de acordo com o autograder:

- Códigos 1 a 4: autograder (12 pontos)
- Códigos 7 e 8: autograder (6 pontos)

Os códigos ID-DFS e LRTA* serão testados pelos monitores sem o uso do autograder. Para a pós-graduação esses dois códigos serão avaliados na computação da nota do EP1. Para a graduação, eles serão avaliados como bônus na média final.